OptSem-C: Public Optimizer Contracts for Query Engine Portability

Haoyi Zhang
Xi'an Jiaotong-Liverpool University
Suzhou, China
hyeliozhang@gmail.com

Huaijin Ran
Nanyang Technological University
Suzhou, China
huaijin003@e.ntu.edu.sg

ABSTRACT

SQL is portable at the language boundary, but optimizer behavior is not. Query engines expose public contracts for pushdown, join ordering, statistics, materialization, distributed placement, runtime adaptation, and plan evidence. Today these behaviors are often compared through coarse labels such as `pushdown`, `join`, or `cost-based`. Such projections collapse local rewrites, source delegation, service-side stages, and runtime decisions into the same apparent capability; in short, coarse optimizer comparison fabricates portability by declaring two engine–query pairs portable even when their public optimizer contracts disagree.

This paper introduces OptSem-C, a formal semantics and benchmark methodology for public optimizer behavior contracts. A contract rule is a guarded modal statement over query features with explicit operator, action variant, modality, optimizer layer, placement, decision time, observability, version, and public evidence. The semantics separates behavioral worlds from evidential support, defines a conflict-aware state join, and characterizes lossy optimizer comparison as a projection kernel over exact contract signatures. On top of this semantics, OptSem-C computes information profiles, exhaustive field-resolution lattices, antichain frontiers, and minimum repair certificates that identify which optimizer-semantic fields are necessary and sufficient for safe comparison.

The grounded study contains 287 verified public-contract rules, 287 source-linked evidence spans, 26 public source records, and 4,216 executable SQL probes covering 7,112 valid optimizer-feature interactions. Keyword projection declares 2,284 equivalences, 254 of which exact contracts refute; operator-only projection declares 2,268 equivalences, 238 of which are false. Exhaustively evaluating all 256 retained-field vocabularies shows that 44 remain unsafe, and after action-variant identifiers are removed the minimum safe semantic vocabularies have size two. Placement repairs keyword collisions, optimizer layer repairs operator-only collisions, and a layer+placement basis repairs all 498 headline witnesses. OptSem-C turns optimizer portability from an informal feature checklist into a finite, auditable query-processing relation.

1 INTRODUCTION

SQL deliberately hides physical execution choices. That abstraction is useful for applications, but it is dangerous for optimizer comparison. The same query can be accepted by several analytical engines while exposing different public behavior: a local predicate rewrite in one engine, connector-side execution in another, a service-side stage graph in a third, and only a logical explanation in a fourth. These differences determine what a benchmark author, system integrator, or database researcher may conclude about join ordering, pushdown, materialization, adaptive execution, and plan evidence before any deployment-specific performance experiment begins.

Database research has precise abstractions for relational semantics, optimizer search, cardinality estimation, adaptive execution, and query equivalence [9, 16, 47–49, 57, 69, 70]. Public optimizer behavior is usually compared with much coarser vocabularies. A cell labeled "pushdown" may mean predicate movement inside the engine, partition pruning, aggregation delegation, join delegation, limit delegation, or runtime filtering. A cell labeled "join" may refer to a logical operator, an estimated physical operator, a distributed exchange, or a runtime profile entry. A claim that two engines are "cost based" may hide whether statistics are local, connector-provided, unavailable, or revised during execution. These shortcuts are not harmless summarization: they create false optimizer portability.

OptSem-C treats this failure as a query-processing problem. A *public optimizer contract* is the optimizer behavior an engine exposes through public plan interfaces, tuning surfaces, and system descriptions. It is not a hidden implementation trace and not a latency measurement. OptSem-C represents each public statement as a guarded modal rule over query features. The rule records the action, the guard under which it applies, where the action is placed, which optimizer layer owns it, when the decision is made, how the behavior is observable, and whether the public contract requires, permits, forbids, or does not specify the action.

The central result is a projection-kernel account of false portability. Comparing optimizer contracts through a keyword or operator vocabulary applies a projection to exact contract signatures. If two exact signatures disagree but fall into the same projected class, the projection has fabricated an equivalence. OptSem-C does more than note that non-injective maps can lose information: it defines the contract object, the evidence-supported signature, the empirical projection kernel, the information retained by each comparison vocabulary, the exhaustive retained-field lattice, and the frontier of minimum semantic repairs. In the grounded study, keyword projection declares 2,284 engine–probe equivalences, 254 of which exact contracts refute. Operator-only projection

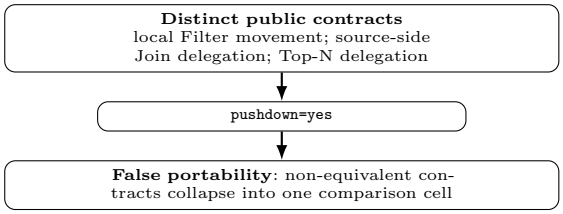

Figure 1: Projection loss in optimizer comparison. A feature-name matrix turns non-equivalent public optimizer contracts into the same apparent capability. OptSem-C preserves the semantic fields that distinguish them.

declares 2,268 equivalences, 238 of which are false. Restoring placement repairs keyword collisions; restoring optimizer layer repairs operator-only collisions; restoring layer and placement repairs all 498 headline witnesses. A field-only stress test over all 256 retained-field vocabularies leaves only 44 unsafe regions, all covered by concrete counter-witnesses; after excluding action-variant identifiers, no singleton semantic field is safe.

We also introduce OptSemBench-C, a covering benchmark for optimizer-contract behavior. Its denominator is not raw query count. It is coverage of optimizer-relevant feature interactions: source boundary, connector capability, join shape, predicate class, aggregation, statistics need, reuse, distribution, adaptivity, and control surface. The generated probes are executable SQL representatives, and the probe space is cross-checked against optimizer-workload motifs from join-order studies, exact-cardinality optimizer testing, cardinality-estimation benchmarks, adaptive query processing, data-lake and federated workloads, star-schema analytics, and optimizer-control surfaces [3, 10, 20, 59, 61, 62, 64, 65].

Contributions. (1) We define public optimizer behavior contracts as a query-processing object distinct from SQL result equivalence and runtime performance. (2) We give guarded modal contract semantics with behavioral worlds, evidence support, state-join algebra, projection kernels, semantic-frontier theorems, and minimum repair certificates. (3) We introduce a covering query-probe benchmark for optimizer-contract feature interactions and connect it to published workload motifs. (4) We construct a verified public-evidence corpus across seven analytical SQL engines. (5) We show that coarse optimizer comparisons fabricate portability across a broad projection portfolio, and that the missing fields are precise query-processing semantics rather than lexical synonyms.

2 THE FAILURE MODE

Figure 1 gives the smallest example. A feature matrix maps several public optimizer contracts into the same label. After the mapping, behaviors with different placement, action kind, and evidence layer become indistinguishable.

Feature-name collision. A cell such as `pushdown=yes` may denote predicate movement inside an engine, partition pruning, connector-side predicate delegation, aggregation delegation, join delegation, limit pushdown, Top-N delegation, or runtime filtering. These actions affect different operators and have different guards, placements, decision times, and observable plan effects. Collapsing them loses the distinctions optimizers use to choose access paths, plan operators, distribution strategies, and reoptimization points [4, 17, 29, 30, 36, 46, 58, 66].

Operator disappearance. A missing native plan node is not a reliable semantics. It may indicate source delegation, logical-plan observability, rewrite, stage-level aggregation, or runtime-only visibility. OptSem-C represents source-delegated execution as a positive contract action rather than as absence. This matters for modern analytical systems whose public surfaces combine local plans, cloud-service stages, connector adapters, and logical explanation outputs [6, 22, 25, 27, 32, 68].

Architecture ranking. A local engine should not be ranked below a distributed engine merely because broadcast, shuffle, or remote-source delegation is not applicable. Conversely, a federated or cloud engine should not be credited with local optimizer behavior when the public contract exposes delegation or service-side stages. OptSem-C separates non-applicability, documented absence, documented optional behavior, and unspecified evidence.

Observable contract versus hidden implementation. The object of study is not what an engine could do internally. It is what the public contract supports as a comparison claim. This distinction is analogous to the separation between SQL equivalence and optimizer-contract equivalence: result-preserving rewrites can be semantically equivalent while exposing different optimizer behavior, different evidence layers, and different portability obligations [1, 8, 13, 14, 72].

These failures imply a minimum payload for comparing public optimizer behavior: guard, canonical action, modality, placement, decision time, observability, version, and provenance. The rest of the paper formalizes this payload and measures what happens when a comparison vocabulary projects it away.

3 CONTRACT SEMANTICS

A query probe is a SQL skeleton paired with a finite feature vector and validity constraints. Features describe optimizer-relevant structure: join shape and type, predicate class, aggregation, order/limit behavior, source boundary, connector capability, statistics need, reuse structure, distribution trigger, adaptivity trigger, and optimizer control surface. They are not workload-frequency estimates; they are coordinates on the public optimizer decision surface.

Definition 1 (Public contract rule). *Let $\mathcal{A}$ be the finite domain of canonical optimizer actions. An action records operator class, action kind, optimizer layer, execution placement, decision time, observability, and variant. A contract state is drawn from $\mathbb{S} = \{\mathsf{MUST}, \mathsf{MAY}, \mathsf{MUST\_NOT}, \mathsf{UNSPEC}\}$. A public contract rule is $r = (\gamma, a, s, \nu, \epsilon)$, where γ is a guard over query*

features, $a \in \mathcal{A}$, $s \in \mathbb{S}$, ν is the version or time window, and ϵ is evidence provenance. For engine e, C_e is the finite rule set, and $C_e(q) = \{r \in C_e \mid \gamma_r(\phi(q)) = \mathsf{true}\}$ is the rule set applicable to probe q.

The state domain has two components that feature matrices usually conflate. Behaviorally, MUST requires an action, MUST_NOT forbids it, and both MAY and UNSPEC leave it possible. Evidentially, MAY is public support for optional behavior, while UNSPEC is lack of public support. Equivalently, a state is interpreted as (m, u, e): required behavior, allowed behavior, and evidence support. This is why documented optional pushdown and undocumented implementation possibility are different public contracts.

For each engine-probe pair, OptSem-C constructs a contract map $M_{e,q} : \mathcal{A} \to \mathbb{S}$, defaulting every action to UNSPEC. Applicable rules update the map with a conflict-aware state join $\sqcup$. The join strengthens unspecified actions when public evidence is found, preserves compatible stronger evidence, and reports contradictions instead of averaging them. The evidence component follows database-provenance practice: support is explicit, inspectable, and attached to the object it justifies [7, 11, 50, 51].

Table 1: State join. U=UNSPEC, N=MUST_NOT, C=CONFLICT.

$\sqcup$	U	MAY	MUST	N
U	U	MAY	MUST	N
MAY	MAY	MAY	MUST	C
MUST	MUST	MUST	MUST	C
N	N	C	C	N

Definition 2 (State-join algebra). *The binary operator $\sqcup$ is the partial join shown in Table 1. It is total over non-conflicting pairs and returns* CONFLICT *for pairs that simultaneously require and forbid, or support and forbid, the same canonical action. Let $x \preceq y$ denote that $x \sqcup y = y$ for non-conflicting states. Then* UNSPEC *is the least informative state, while* MUST *and* MUST_NOT *are incomparable incompatible commitments.*

Lemma 1 (Merge invariants). *On non-conflicting states, $\sqcup$ is commutative, associative, idempotent, and monotone with respect to $\preceq$. Therefore a contract map obtained by joining applicable rules is independent of rule order.*

Proof. The properties follow by inspection of the finite table. Idempotence holds because every diagonal entry is the input state. Commutativity holds because the table is symmetric. Associativity and monotonicity are checked over the finite non-conflicting triples; the only excluded triples are exactly those containing incompatible positive and negative commitments. Lifting the pointwise operator to maps preserves the properties for every action independently. □

Let $W \subseteq \mathcal{A}$ be an abstract optimizer-behavior world. A map M denotes the world set $[\![M]\!] = \{W \mid M(a) = \mathsf{MUST} \Rightarrow a \in W,\ M(a) = \mathsf{MUST\_NOT} \Rightarrow a \notin W\}$. The evidential support set is $\mathsf{Supp}(M) = \{a \in \mathcal{A} \mid M(a) \in \{\mathsf{MUST}, \mathsf{MAY}, \mathsf{MUST\_NOT}\}\}$. Behavioral containment and evidential support are intentionally separate: two maps can allow the same worlds while differing on whether an action is publicly documented.

Definition 3 (Exact signatures, kernels, and projections). *The exact evidence-supported signature of M is $S(M) = \{(a, M(a)) \mid a \in \mathsf{Supp}(M)\}$. Exact equivalence is equality of signatures over full evidence atoms. A projection is a declared map $\pi : \mathcal{A} \times \mathbb{S} \to \mathcal{B}$ from full evidence atoms to a comparison vocabulary such as keyword, yes/no, or operator-only labels. The projected signature is the finite image $S_\pi(M) = \{\pi(a, M(a)) \mid a \in \mathsf{Supp}(M)\}$. The projection kernel is the equivalence relation $\mathcal{K}_\pi$ where $M_i \mathcal{K}_\pi M_j$ iff $S_\pi(M_i) = S_\pi(M_j)$. A false portability witness is a pair in $\mathcal{K}_\pi$ whose exact signatures differ.*

For quantitative comparison, exact compatibility is the Jaccard overlap $|S(M_i) \cap S(M_j)|/|S(M_i) \cup S(M_j)|$, and projected compatibility applies the same formula to S_π. The denominator is evidence-supported: undocumented actions do not count as evidence of either equality or inequality.

Theorem 1 (Finite evaluation). *For finite feature and action domains, constructing $M_{e,q}$ from finite C_e and q is decidable.*

Proof. Every guard is a finite Boolean formula over a finite feature vector. The applicable rule set is finite, and every applicable rule performs one lookup in the finite state-join table. Construction therefore terminates. □

Theorem 2 (Projection-kernel loss). *For any projection π, false portability is exactly the nontrivial part of the kernel $\mathcal{K}_\pi$: M_i and M_j are falsely portable iff $S(M_i) \neq S(M_j)$ and $M_i \mathcal{K}_\pi M_j$. If π is non-injective on evidence atoms, there exist contracts with a false-portability witness.*

Proof. The first statement unfolds the definitions: $\mathcal{K}_\pi$ is equality after projection, while exact non-equivalence is $S(M_i) \neq S(M_j)$. For existence, take distinct evidence atoms $(a_1, s_1) \neq (a_2, s_2)$ with $\pi(a_1, s_1) = \pi(a_2, s_2)$. Let M_1 contain only (a_1, s_1) and M_2 contain only (a_2, s_2). Their exact signatures differ, but their projected signatures are the same singleton. □

Definition 4 (Projection information profile). *For a finite set of observed maps Y, let $H(S)$ be the empirical Shannon entropy of exact signatures and $H(S_\pi)$ the entropy of projected signatures. The entropy-retention ratio $\rho_\pi = H(S_\pi)/H(S)$ measures distinguishability retained by the comparison vocabulary. The projected collision count is the number of projected-signature classes containing more than one exact signature class.*

Lemma 2 (Kernel inflation). *If a projection has projected-equivalence count E_π and exact-equivalence count E over a fixed pair set, then E_π/E is the pairwise inflation factor induced by the kernel. Every false-equivalence witness*

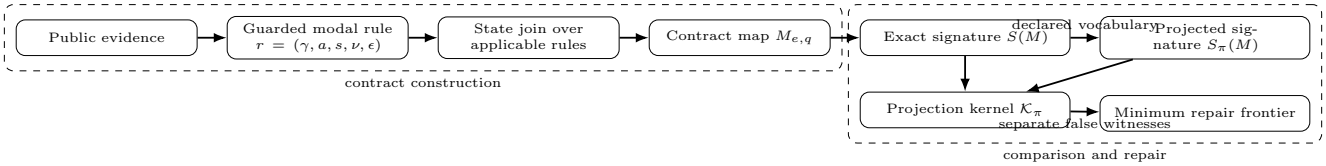

Figure 2: Contract-comparison pipeline. Public evidence becomes guarded modal rules, rules merge into per-engine/per-probe contract maps, exact signatures are projected into a declared comparison vocabulary, and nontrivial projection kernels yield false-portability witnesses and minimum repair frontiers.

contributes to this inflation, and a zero-false strengthened projection must have $E_\pi = E$.

Proof. Exact equivalence implies projected equivalence only when the projection is a function of the exact atom payload, so $E_\pi \geq E$. The additional pairs counted by E_π are exactly the unequal exact signatures that collapse under π, which are false-portability witnesses. If there are none, no additional pairs are counted and the two denominators agree. □

Theorem 3 (Evidence refinement). *Adding a non-conflicting evidence-backed rule cannot silently weaken a public contract: it behaviorally refines possible worlds, evidentially refines support, or both.*

Proof. A MUST or MUST_NOT rule removes worlds from $[\![M]\!]$. A MAY rule may leave worlds unchanged, but changes an action from UNSPEC to evidence-supported. Because the rule is non-conflicting, the state join does not erase existing support or replace a compatible stronger state with a weaker one. □

A projection error is therefore a collision, not a statistical residual. Let F be the semantic-field universe and let π_R be the repaired projection that appends the field values in $R \subseteq F$ to $\pi(a, s)$ for each evidence atom (a, s). For a finite grounded comparison set X, define $X_{\pi_R} = \{(M_i, M_j) \in X \mid S(M_i) \neq S(M_j) \wedge M_i \mathcal{K}_{\pi_R} M_j\}$. A field set R is safe, or universal, for π on X when $X_{\pi_R} = \emptyset$; otherwise it is unsafe and has a grounded counter-witness.

Theorem 4 (Projection-lattice frontier). *If $R \subseteq R' \subseteq F$, then $X_{\pi_{R'}} \subseteq X_{\pi_R}$. Consequently the safe field sets are upward closed, the unsafe field sets are downward closed, and the minimum universal repairs form an antichain frontier in the finite field lattice.*

Proof. Projection $\pi_{R'}$ emits every component emitted by π_R plus additional field values. Adding fields can split projected equivalence classes, but cannot merge classes already separated. Therefore any unequal pair still collapsed after adding R' was also collapsed after adding R. Upward closure of safe sets and downward closure of unsafe sets follow immediately, and minimal safe sets are incomparable by set inclusion. □

For a witness $w = (M_i, M_j)$, define its separator family $\mathcal{D}_w = \{R \subseteq F \mid M_i \not\mathcal{K}_{\pi_R} M_j\}$. By Theorem 4, each $\mathcal{D}_w$ is upward closed in the field lattice, and universal repair is the finite intersection problem $R \in \bigcap_{w \in X_\pi} \mathcal{D}_w$.

Theorem 5 (Repair certificate). *Fix a projection π and finite comparison set X. A field set R eliminates all observed false equivalences iff R belongs to every witness separator family $\mathcal{D}_w$ for $w \in X_\pi$.*

Proof. If $R \in \mathcal{D}_w$ for every witness, every pair collapsed by π becomes separated under π_R, so $X_{\pi_R} = \emptyset$. Conversely, if $X_{\pi_R} = \emptyset$, every original witness is separated under π_R, which is exactly membership in each separator family. □

Theorem 6 (Resolution-lattice certificate). *For a finite comparison set X and finite field universe F, exhaustive enumeration of all retained-field projections over $2^{|F|}$ subsets is a complete certificate of which public optimizer fields are sufficient for safe comparison on X. If a subset R is unsafe, a single pair $(M_i, M_j) \in X$ with $S(M_i) \neq S(M_j)$ and $S_R(M_i) = S_R(M_j)$ is a checkable counter-certificate. If R is safe, direct equality testing over all pairs in X is a checkable certificate of safety.*

Proof. For each subset $R \subseteq F$, the retained-field signature S_R is a deterministic projection of the exact signature. The comparison set X is finite, so checking all pairs in X terminates. Unsafety is existential and is witnessed by one collapsed unequal pair. Safety is universal over the same finite set and is witnessed by the absence of such a pair after exhaustive enumeration. Since every subset of F is enumerated, the resulting safe and unsafe regions are complete for the chosen field universe. □

Theorem 7 (Minimum repair as hitting set). *For finite semantic fields F and false witnesses X_π, deciding whether there is a universal repair of size at most k is NP-complete in $|F| + |X_\pi|$. For the fixed OptSem-C field universe, every minimum repair is exactly computable, and a certificate consists of the repaired witnesses plus counter-witnesses for every smaller field set.*

Proof. Membership in NP follows by checking a candidate field set against every finite witness. For hardness, reduce Hitting Set. Given sets $H_1, \ldots, H_m \subseteq F$, create one false-equivalence witness per H_i whose two contracts agree under the baseline projection and differ exactly on the fields in H_i. A field set repairs that witness iff it intersects H_i; hence a universal repair of size k exists iff the hitting-set instance

has a hitting set of size k. Since OptSem-C fixes a finite field universe, enumerating field subsets in increasing cardinality returns all minimum repairs. Exhausting all smaller subsets gives the lower-bound part of the certificate. □

Theorem 8 (Semantic repair basis). *Fix a finite comparison set X, projections Π, and a semantic field set F that excludes action identifiers such as fine-grained variants. If $R \subseteq F$ is universal for every $\pi \in \Pi$, then augmenting each projection with R eliminates all observed false equivalences without relying on action-identifier labels.*

Proof. Apply Theorem 5 to each projection independently. Because X and Π are finite, the union of their witness sets is finite; a common R that lies in every corresponding separator family leaves no collapsed unequal pair. Excluding action identifiers restricts the admissible field universe but does not change the certificate condition. □

The semantics makes the comparison payload explicit. A user comparing only operator names chooses a projection that drops placement, decision time, variant, and observability. A feature checklist chooses an even coarser projection that can drop operator class itself. OptSem-C does not forbid those projections; it computes exactly which kernel collisions they induce, which grounded counter-witnesses remain, and which query-processing fields repair the comparison.

4 BENCHMARK AND GROUNDED CORPUS

OptSemBench-C is a behavior-contract benchmark. It does not measure latency or rank engines. It generates probes that cover optimizer-relevant feature interactions and then measures which public contracts those probes activate. This follows the benchmark principle that the metric and denominator must match the object of study: plan-quality benchmarks ask whether cardinality estimates improve plans, data-lake benchmarks expose explicit discovery denominators, and optimizer-contract benchmarks must expose semantic traps rather than only query counts [12, 23, 34, 38, 53, 63, 73, 75].

Let $F = F_1 \times \cdots \times F_n$ be the finite feature domain and Ψ the validity constraints. A target interaction is a valid partial assignment over a feature subset. The benchmark targets global pairwise coverage plus selected three-way interactions for source boundary, connector capability, operator class, statistics need, join shape, adaptivity trigger, and distribution trigger. A greedy generator selects renderable full assignments until all renderable target interactions are covered. The generator has two responsibilities. First, it separates benchmark adequacy from any particular system: adequacy is measured against the feature-interaction universe, not against runtime results. Second, it produces probes that are diagnostic rather than workload-frequency estimates. A probe is useful when it activates a contract guard or separates two contract signatures.

Theorem 9 (Coverage). *If every valid target interaction has at least one renderable full assignment, the generator terminates with a probe set covering every renderable target interaction.*

Proof. The target universe is finite. Each selected assignment covers at least one uncovered interaction. Thus the number of uncovered interactions decreases monotonically until it reaches zero. □

Grounded and generated object	Value
Verified public contract rules	287
Evidence spans / source records	287 / 26
Generated SQL probes	4,216
Valid optimizer-feature interactions	7,112
Rules triggered by probes	287 / 287
SQL probes executed	4,216 / 4,216
External motifs covered and executed	90 / 90
Audit / merge / shape / execution issues	0 / 0 / 0 / 0

Design invariant	What it prevents
Ground first	Unsupported optimizer folklore entering the mainline corpus.
Cover then execute	Feature-complete but non-runnable benchmark probes.
Separate then repair	Counting false portability without a semantic explanation.
Crosswalk externally	Treating a private feature grid as a benchmark substitute.

Figure 3: Corpus, benchmark, and execution denominators. Every mainline rule is public-evidence grounded and every generated SQL probe executes in the validation catalog.

The main empirical corpus contains only verified grounded rules. Each rule is linked to a public evidence span, source URL, source identifier, line range, evidence digest, and source class. Rules without source-grounded evidence are excluded from the main results. The grounded corpus covers plan observability, pushdown, join search, distribution, materialization, runtime adaptivity, statistics, and optimizer control surfaces. Every grounded rule is triggered by at least one generated probe, and contract merge produces no conflicts.

The source set includes public references for all engines in the study. Examples include PostgreSQL plan and query-planning documentation, Trino cost-based optimization and pushdown documentation, BigQuery stage and query-plan documentation, Spark SQL adaptive-execution documentation, Snowflake logical-plan documentation, DuckDB CTE materialization documentation, and ClickHouse join-algorithm documentation [15, 42, 43, 45, 52, 55, 56]. These sources are not used as performance measurements. They provide the public contracts that the framework formalizes.

The grounded corpus is intentionally conservative. A rule is admitted only when the evidence span supports an optimizer action, a guard or applicability condition, and an observability or placement interpretation. Vague performance advice is excluded from the main corpus. This policy reduces corpus size but strengthens the central claim: the evaluation is about contracts that are publicly supported, not about inferred implementation behavior.

The guard audit in Table 3 checks that this conservatism does not create unreachable contracts. A finite-disjunction guard semantics evaluates each admitted rule against the

Table 2: Representative source-grounded contract examples. The main corpus stores one source-linked evidence span per accepted rule; this table shows the kind of public evidence that becomes a contract rule.

Engine	State	Canonical action	Source lines	Evidence-backed contract paraphrase
PostgreSQL	MAY	Plan/observe/local	L46–L56	EXPLAIN exposes estimated startup and total cost, output rows, row width, and planner-cost units.
BigQuery	MAY	Exchange/adapt/cloud	L990–L992	The query plan may be modified during execution by adding repartition stages.
Snowflake	MAY	Plan/observe/cloud	L150–L154	EXPLAIN compiles a statement and returns a logical execution plan rather than executing it.
ClickHouse	MAY	Join/adapt/local	L313	With automatic join-algorithm selection, execution can switch from hash join to partial merge join.
DuckDB	MAY	Materialize/inline/local	L482–L487	CTE inlining is guarded by single use, absence of volatility, no grouped aggregation, and inlining hints.
Spark SQL	MAY	Plan/adapt/distributed	L129–L133	Adaptive query execution uses runtime statistics and can re-optimize a query while it is running.
Trino	MUST_NOT	Aggregate/pushdown/source	L168–L176	Aggregation pushdown does not support aggregate calls containing expressions.

Table 3: Guard-quality audit for grounded public-contract rules. Probe support uses finite-disjunction guard semantics; non-global rules depend on at least one query feature.

Audit	Value	Meaning
Grounded rules	287	finite public-contract relation
Rules with probe support	287/287	every admitted rule is triggerable
Guarded non-global rules	252	query-conditional contracts
Median probe support	102	narrow-to-global guard range
Same-action containments	3	finite overlap explicitly audited
Invalid guard dimensions	0	guards stay inside feature domain

Table 4: Executable query-probe corpus. The benchmark contains the full generated SQL corpus and a compact motif-cover prefix for human-scale inspection.

Quantity	Value
Generated SQL probes	4,216
Representative motif-cover probes	48
Shape-invalid rendered probes	0
Corpus digest prefix	`df107f018e37`

generated probe denominator. All 287 grounded rules have at least one triggering probe, 252 are non-global query-conditional rules, and no guard uses a feature dimension or value outside the benchmark domain.

Coverage is reported over two denominators. The first is benchmark coverage: every targeted renderable valid feature interaction is covered by at least one generated probe. The second is contract triggering: every grounded rule is applicable to at least one probe. Together these denominators make the empirical scope explicit. A rule that cannot be triggered by any probe would indicate a benchmark gap; a feature interaction that cannot be rendered is excluded from the coverage denominator and reported as a modeling constraint.

The benchmark also exposes executable SQL shapes. Each generated probe has a normalized SQL skeleton and shape

Table 5: Full executable SQL validation of generated probes. The validation catalog is deterministic and is used only to check SQL shape and planability, not to rank engine performance.

Quantity	Value
Generated SQL probes	4,216
Probes with query plan	4,216
Executable probes	4,216
Execution failures	0
Distinct validation plans	89
Returned rows	16,988

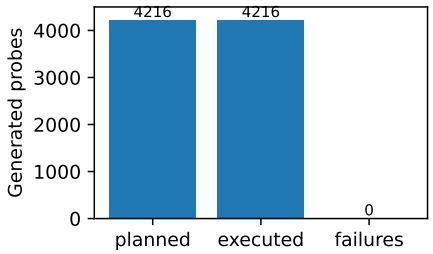

Figure 4: Executable SQL-probe validation. Every generated probe plans and executes on the deterministic validation catalog; failures are zero.

checks for joins, filters, grouping, ordering, limits, and reuse structure. The full 4,216-probe corpus is accompanied by a compact 48-probe motif-cover prefix. The prefix is not a replacement for the full evaluation; it is a human-scale representative set that covers every external motif family.

A common failure mode in generated benchmarks is to produce feature-complete but non-executable query text. OptSemBench-C therefore validates every SQL skeleton on a deterministic relational catalog with local, remote, and cross-source namespaces. This validation is not a performance

Table 6: SQL validation across deterministic catalog densities. The check repeats the full generated probe corpus over three populations; it validates query construction and planability, not runtime performance.

Rows/table	Probes	Exec.	Fail	Rows
1	4,216	4,216	0	3,802
5	4,216	4,216	0	16,988
17	4,216	4,216	0	48,969

Table 7: Real-engine validation of generated SQL probes. The check records parse, plan, execution, visible plan tags, and plan-hash diversity; it is not a latency benchmark.

Engine	Scope	Probes	Exec.	Fail	Mean tag match
DuckDB	Full	4,216	4,216	0	0.705
PostgreSQL	Full	4,216	4,216	0	0.796
DuckDB	Motifs	71	71	0	0.756
PostgreSQL	Motifs	71	71	0	0.819

baseline. Its role is stricter and more basic: every generated probe must be parseable, plannable, executable, and shape-consistent with its feature vector. Table 5 and Figure 4 report the validation result: all 4,216 generated probes obtain a plan and execute, with zero SQL-shape failures. Table 6 repeats the full corpus over three deterministic catalog populations. All 12,648 probe-catalog executions succeed, so the validation is not a single-population effect. Table 7 adds a production-engine check: the same generated SQL corpus is parsed, planned, and executed on DuckDB and PostgreSQL. Across 8,432 full-corpus engine–probe executions and 142 motif-representative executions, both engines report zero execution failures. The visible tag match is intentionally reported separately from execution success because real optimizers may satisfy a feature without printing every expected tag in the textual plan.

The same fixed probes support two budgeted evaluation orders. The cover order is the generator's feature-interaction order. The diagnostic order greedily maximizes newly triggered grounded rules without adding any new probe. Table 8 reports both. The cover order triggers all grounded rules within 100 probes; the diagnostic order triggers 286 of 287 rules within 50 probes and all rules within 100. Random subsets of 100 probes trigger 81% of rules on average and 85% at the 95th percentile. Thus OptSemBench-C is not merely large enough to cover the surface; it also contains compact diagnostic prefixes that expose almost all grounded contracts.

A benchmark for public optimizer contracts must connect to existing database workloads without becoming another latency benchmark. We therefore add a crosswalk from published optimizer and benchmark motifs to OptSemBench-C feature requirements: TPC-H and TPC-DS decision-support structure, JOB join-order sensitivity, SSB star-schema analytics, cardinality-estimation plan quality, adaptive replanning,

Table 8: Contract-trigger efficiency for two replay orders over the same fixed probe set. Random intervals are 5th–95th percentile over 100 same-budget subsets.

Budget	Cover	Diagnostic	Random mean	Random 5–95
50	.672	.997	.707	[.627,.767]
100	1.000	1.000	.811	[.770,.850]
250	1.000	1.000	.890	[.864,.913]
500	1.000	1.000	.920	[.899,.941]
1000	1.000	1.000	.951	[.934,.969]
2000	1.000	1.000	.973	[.958,.986]
4216	1.000	1.000	1.000	[1.000,1.000]

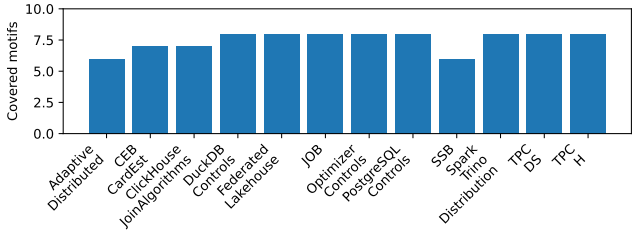

Figure 5: External optimizer-motif denominator. The generated probe space covers 90 motifs across 12 recognizable optimizer and benchmark families.

Table 9: Crosswalk from published optimizer/benchmark motifs to OptSemBench-C feature requirements.

Motif	Requirements	Covered
Join-order sensitivity	8	8
Exact-cardinality testing	9	9
CardEst plan quality	8	8
Adaptive replanning	9	9
Cross-source joinability	7	7

cross-source/data-lake joinability, PostgreSQL and DuckDB optimizer controls, ClickHouse join algorithms, Spark and Trino distribution controls, and user-visible optimizer-control surfaces [2, 5, 18, 19, 26, 33, 37, 54, 67, 71]. Table 9 reports the executable check. All 90 declared motif requirements across 12 families are covered by the generated probe space; the crosswalk is a coverage audit over optimizer-decision surfaces, not a copy of third-party benchmark SQL.

The motif-difficulty view prevents a misleading denominator. Some motifs are broad and match many probes, such as star-schema group-by shapes. Others are sparse, such as a TPC-H conjunctive-filter motif that has one matching rendered probe under the current validity constraints. Reporting both coverage and matching-probe counts makes benchmark adequacy inspectable: coverage says the motif is present, while difficulty says how concentrated or redundant that coverage is. The executable motif suite adds a second validation: for each external motif, the system selects one satisfying probe and validates the generated SQL on the deterministic catalog. Thus the external denominator is not a list of names; it is a set of optimizer-decision requirements with concrete

Table 10: External motif difficulty. Matching probes are generated OptSemBench-C probes satisfying each benchmark-family motif requirement.

Suite	Motifs	Min	Median	Max
TPC-H	8	1	87.0	1384
TPC-DS	8	86	164.5	287
JOB	8	55	87.0	236
SSB	6	89	101.0	3442
CardEst	7	72	228.0	259
Adaptive	6	72	205.5	235
Federated	8	72	91.5	633
Controls	8	70	72.0	73
PostgreSQL	8	72	73.0	73
DuckDB	8	68	72.0	364
ClickHouse	7	70	70.0	70
Spark/Trino	8	69	70.0	236

Table 11: External benchmark-motif representative validation. Each motif is mapped to a generated probe and validated on the deterministic catalog.

Suite	Motifs	Validated	Result rows
TPC-H	8	8	28
TPC-DS	8	8	34
JOB	8	8	28
SSB	6	6	22
CEB-CardEst	7	7	31
Adaptive-Distributed	6	6	30
Federated-Lakehouse	8	8	40
Optimizer-Controls	8	8	40
PostgreSQL-Controls	8	8	35
DuckDB-Controls	8	8	20
ClickHouse-JoinAlgorithms	7	7	35
Spark-Trino-Distribution	8	8	40
Total	90	90	383

SQL representatives. Engine-plan validation is a separate experiment because the motif suite itself is a denominator check rather than a claim that external benchmark queries and generated probes induce identical plans.

5 EVALUATION

The evaluation asks four questions. First, when a coarse optimizer comparison declares two public contracts equivalent, how often is that equivalence false? Second, which semantic fields repair the false equivalence? Third, do the observed failures correspond to recognizable query-processing mechanisms rather than corpus-construction effects? Fourth, how much distinguishability does each comparison vocabulary retain?

Coarse comparison fabricates equivalence. Table 12 reports the main measurement. The keyword projection declares 2,284 equivalences, including 254 false ones. The operator-only projection declares 2,268 equivalences, including 238 false ones. These errors are conditional on the baseline declaring equivalence, which is the decision faced by a user of a feature matrix. The results are stable under binomial and probe-cluster uncertainty estimates, and false equivalence remains nonzero for keyword and operator-only projections after removing any single public source. A strict-projection negative control yields zero false equivalences.

Table 13: Severity of fabricated equivalence. Exact distance is Jaccard distance over evidence-supported contract signatures among pairs that a projection declares equivalent but exact contracts refute.

Projection	False	Mean dist.	Mean atom Δ	Max atom Δ
Keyword	254	1.00	9.09	17
Yes/No	6	1.00	3.00	3
Operator	238	1.00	5.84	14
Placement	10,702	1.00	21.29	53
State	23,593	1.00	13.06	64
Op+kind+surface	0	0.00	0.00	0

Table 12: False portability and repair summary. Conditional false equivalence is measured only among pairs that the projection declares equivalent.

Projection	Eq.	False	Cond.	Repair field(s)
Keyword	2,284	254	11.1%	placement
Yes/No	2,036	6	0.3%	layer
Operator-only	2,268	238	10.5%	layer

Counts alone understate the failure. Table 13 measures how far apart the exact contracts are after a projection has declared them equal. For the headline projections, every false-equivalent pair has exact-signature Jaccard distance 1.0: the projected comparison has equated disjoint public contract signatures rather than near duplicates. Keyword false equivalences differ by 9.09 evidence atoms on average, and operator-only false equivalences differ by 5.84 atoms. The strengthened surface projection is the control: it returns to the exact-equivalence denominator and has zero fabricated equivalence.

Table 14 makes the baseline portfolio explicit. The baseline portfolio evaluates exact negative controls, common matrices, one-field adversarial baselines, and strengthened high-information baselines. The exact-contract baseline produces no false equivalence. Keyword, kind-only, and operator-only comparisons fabricate false equivalence at similar rates, so the result is not tied to one synonym list. One-field baselines such as placement-only, decision-time-only, and state-only are much worse, showing that comparing a single optimizer dimension is not a safe substitute for contract semantics. By contrast, the strengthened operator/action/layer/placement/observability baseline has zero false equivalence, which localizes the missing semantics rather than merely declaring all coarse comparisons bad.

Projection kernels are measurable objects. Table 15 reports the finite information profile of representative projections. Exact signatures form 1,288 observed classes. Keyword projection compresses them into 148 projected classes, 81 of which contain multiple exact classes, retaining 64% of empirical signature entropy and inflating declared equivalence by 12.5%. Operator-only projection retains more entropy but still leaves 191 colliding projected classes and 238 false equivalences. Placement-only and state-only baselines demonstrate the opposite failure mode: a single field creates very large kernels. The strengthened surface projection still has

Table 14: Executable projection baselines. False equivalence is conditional on the baseline declaring equivalence; the full baseline portfolio contains 17 projections.

Baseline	Eq.	False	Rate
Exact	2,030	0	0.0%
Keyword	2,284	254	11.1%
Op/action	2,036	6	0.3%
Operator	2,268	238	10.5%
Kind-only	2,282	252	11.0%
Layer-only	2,479	449	18.1%
Place-only	12,732	10,702	84.1%
State-only	25,623	23,593	92.1%
Op+kind+surface	2,030	0	0.0%

Table 15: Projection-kernel information profile. Entropy is computed over observed engine-probe contract signatures; false equivalence is pairwise.

Projection	Classes	Colliding	Ent. kept	Eq.	False
Exact	1,288	0	1.000	2,030	0
Keyword	148	81	0.638	2,284	254
Operator	387	191	0.787	2,268	238
Placement	9	8	0.280	12,732	10,702
State	7	5	0.217	25,623	23,593
Op+kind+surface	820	206	0.904	2,030	0

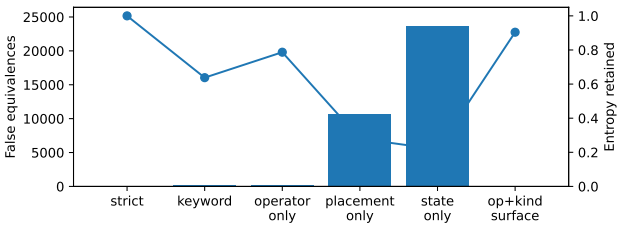

Figure 6: Projection information profile. Bars show false equivalences induced by representative projections; the line shows empirical signature entropy retained by the projection.

projected collisions at the signature-class level, but it does not produce any pairwise false equivalence over the engine-probe comparison set. This is the empirical counterpart of the projection-kernel theorem: the kernel is finite, populated, and repairable.

Resolution-lattice stress test. The information profile asks how much a named projection compresses exact signatures. Table 16 asks a stricter question: if a comparison is allowed to retain any subset of public optimizer fields, which retained-field vocabularies are safe? The test enumerates all 256 subsets of the eight evidence-atom fields and evaluates them over the same 88,536 engine–probe comparisons. The empty vocabulary fabricates 59,546 equivalences. Every singleton semantic field is unsafe: operator alone leaves 238 false equivalences, kind alone leaves 252, layer alone leaves 449, and placement alone leaves 10,702. Excluding action-variant identifiers, the semantic field lattice has 128 subsets, 44 unsafe regions, and minimum safe field count two; the two minimum safe no-variant vocabularies are operator+layer and kind+placement. A compact set of ten concrete engine–probe

Table 16: Exhaustive field-resolution lattice. The baseline retains only the listed public optimizer fields and no engine or probe identifiers; rows without the action-variant identifier show the semantic no-variant frontier.

Retained fields	$\|R\|$	Classes	Ent.	Eq.	False
None	0	1	-0.000	61,576	59,546
Operator	1	387	0.787	2,268	238
Action kind	1	183	0.661	2,282	252
Layer	1	52	0.552	2,479	449
Placement	1	9	0.280	12,732	10,702
Operator+layer	2	591	0.850	2,030	0
Kind+placement	2	214	0.680	2,030	0
Op+kind+layer+placement+obs.	5	820	0.904	2,030	0
All semantic fields	7	947	0.934	2,030	0

Table 17: False-portability robustness. Micro rates condition on the projection declaring equivalence; pair macro averages within engine pairs. Leave-one-source ranges remove all rules supported by one public source.

Projection	Decl.	False	Micro	Pair macro	LOO range
Keyword	2284	254	0.111	0.402	[0.017,0.144]
Yes/No	2036	6	0.003	0.003	[0.000,0.052]
Operator-only	2268	238	0.105	0.290	[0.032,0.148]

witnesses covers all unsafe retained-field regions, so the frontier is not driven by one example row. Deriving the frontier antichains gives 14 minimal safe no-variant vocabularies and eight maximal unsafe vocabularies; the monotone frontier certificate shows that every safe retained-field vocabulary contains one of the minimal safe elements and every unsafe vocabulary is contained in a maximal unsafe element.

The robustness view in Table 17 checks that the headline result is not an effect of a single denominator. The micro rate conditions on the baseline declaring equivalence; the pair-macro rate averages within engine pairs; and the leave-one-source range removes all rules supported by one public source. Keyword and operator-only false portability remain visible across these views, including every leave-one-source ablation for those two projections.

The evidence side of the same claim is also cross-checked. Every headline false-portability witness is supported by at least two distinct public source records; keyword and operator-only witnesses draw on 11 and 8 source records. A side-balanced audit further shows that all 498 headline witnesses have public support on both compared signatures, with no left/right source reuse, so the kernel is not a one-sided evidence effect.

A second overfitting check asks whether the false witnesses are concentrated in one hand-picked query shape. Table 18 shows the opposite. The 498 headline false witnesses occupy 486 distinct query probes. Keyword and operator-only witnesses are almost one-per-probe, touch every feature dimension, and cover 92 and 75 observed feature values respectively. The witnesses are therefore a dispersed projection-kernel phenomenon rather than a duplicated-row effect.

Table 18: False-witness dispersion. A witness is one engine–probe pair declared equal by a projection and unequal by exact contracts.

Projection	Witnesses	Probes	Pairs	Values	Max/probe
Keyword	254	254	3	92	1
Operator-only	238	238	2	75	1
Yes/No	6	6	1	15	1

Table 19: Finite-comparison scalability and dispersion. False probes and feature axes summarize where false witnesses occur; family regions are pair-family groups with at least one false witness. The 8× lift repeats the probe denominator without changing contracts. Drift compares streaming maintenance with full recomputation.

Projection	False	Probes	Axes	Fam.	Min checks/s	Drift
strict	0	–	–	0/6	257,227	0
keyword	254	254	9	3/6	321,414	0
operator-only	238	238	8	2/6	337,362	0
surface	0	–	–	0/6	265,000	0

Table 19 adds stress views that are easy to miss in a small feature matrix. The finite comparison audit executes all 88,536 engine–probe pair checks for each projection, so the reported kernel is not sampled. A deterministic 8× lift reaches 708,288 signature comparisons per projection while preserving the same exact-contract evidence; all tested projections sustain above 70,000 comparisons per second, and prefix-scaling regressions have $R^2 \geq 0.96$. A streaming maintenance audit inserts probes one at a time and independently recomputes each prefix; the drift is zero for every projection, and the full prefix uses 88,536 comparison work units rather than the 186.7 million work units implied by recomputing every prefix from scratch. The table also ties scaling to dispersion and family stratification: keyword false portability spans 254 probes, nine feature axes, and three of six pair-family regions; operator-only false portability spans 238 probes, eight feature axes, and two regions. Removing one engine or one engine family never creates an unresolved witness after the layer+placement repair. For public evidence refreshes, the largest source-local update touches one engine's probe map, and no source contributes more than 16.1% of grounded rules.

The probe-subsample view in Table 20 checks that the false-portability result is not an effect of using the full generated benchmark. At 10% and 50% probe subsamples, keyword and operator-only projections remain false in every trial, with median conditional rates close to the full benchmark. The weaker yes/no projection is sparse at 10%, but becomes nonzero in every trial by 50%.

The mutation-suite view is adversarial: it expands the comparison vocabulary around the headline baselines rather than only evaluating the three projections used in the abstract. Exact signatures remain a zero-false negative control, one-field projections produce much larger kernels, and strengthened projections that restore operator, action kind, and the public semantic surface collapse to the exact-equivalence denominator. The result is not that every abstraction is bad; it is

Table 20: Probe-subsample robustness. Rates are conditional false-equivalence rates; nonzero reports trials with at least one false equivalence.

Projection	Probes	Median rate	Range	Nonzero
keyword	10%	0.107	[0.074, 0.165]	30/30
keyword	50%	0.111	[0.096, 0.126]	30/30
keyword	100%	0.111	[0.111, 0.111]	1/1
operator-only	10%	0.102	[0.056, 0.146]	30/30
operator-only	50%	0.102	[0.089, 0.117]	30/30
operator-only	100%	0.105	[0.105, 0.105]	1/1
yes/no	10%	0.002	[0.000, 0.011]	15/30
yes/no	50%	0.003	[0.001, 0.006]	30/30
yes/no	100%	0.003	[0.003, 0.003]	1/1

Table 21: Repair fields and attribution agree.

Projection	Single-field repairs	Top attribution fields
Keyword	variant, placement	variant=.168; placement=.168
Yes/No	variant, layer, placement, state	each=.250
Operator-only	variant, layer	variant=.184; layer=.184

that common portability abstractions omit query-processing fields that several independently weakened projections need and several strengthened projections restore.

Repair certificates identify the missing semantics. A projection can be repaired by restoring semantic fields. The repair column in Table 12 reports minimum-cardinality universal repairs and held-out-probe generalization. Placement repairs every keyword false equivalence. Layer repairs every operator-only false equivalence. The learned repairs resolve all held-out false equivalences across probe folds. A fixed semantic basis consisting of optimizer layer and execution placement repairs all observed false equivalences across keyword, yes/no, and operator-only projections, even when fine-grained action-variant labels are excluded. The certificate is checked as a finite proof obligation: each counted witness is full-signature unequal, projection-equal, separated by the reported repair, and protected by a grounded counter-witness against every smaller universal repair. The hitting-set view over all 498 false witnesses agrees exactly with direct repaired-signature enumeration.

Table 23 adds a stronger anti-overfitting test. Each fold withholds a non-default query-feature family, learns repairs from the remaining false witnesses, and tests them on the held-out family. A point minimum can choose an underspecified field when the training denominator is narrow; the fixed semantic basis layer+placement resolves every held-out false witness across all folds. The repair is therefore not a post-hoc fit to one query slice, but a stable contract denominator.

The repair ladder in Table 22 gives a stepwise view of the same diagnosis. Restoring operator alone nearly repairs keyword projection but leaves six false equivalences; restoring layer alone reduces but does not eliminate keyword false

Table 22: Projection repair ladder. Rows show how false equivalence changes as semantic fields are restored to a coarse projection.

Projection	Restored fields	Eq.	False
Keyword	–	2,284	254
Keyword	operator	2,036	6
Keyword	layer	2,072	42
Keyword	placement	2,030	0
Keyword	layer+placement	2,030	0
Yes/No	–	2,036	6
Yes/No	layer	2,030	0
Yes/No	placement	2,030	0
Operator	–	2,268	238
Operator	kind	2,036	6
Operator	placement	2,062	32
Operator	layer	2,030	0
Operator	layer+placement	2,030	0

Table 23: Feature-family held-out repair stability. Each fold withholds a non-default query-feature family, learns repairs on the remaining false witnesses, and evaluates held-out witnesses. The robust semantic basis is fixed before testing.

Projection	Folds	Held-out false	Point-min unresolved	Robust unresolved
Keyword	12	830	144	0
Operator-only	9	569	12	0
Yes/No	1	3	0	0

Table 24: Semantic repair frontier over the core field universe. Safe subsets repair all false equivalences for the projection scope; unsafe subsets leave at least one grounded counter-witness.

Scope	False	Safe	Unsafe	Min.	Example minimum
Keyword	254	23	9	1	placement
Yes/No	6	24	8	1	placement
Operator-only	238	22	10	1	layer
All	498	21	11	2	layer+placement

equivalence; restoring placement eliminates it. For operator-only projection, restoring layer eliminates all false equivalences while placement alone leaves residual errors. This ladder makes the repair claim auditable rather than merely descriptive.

Table 24 reports the field-lattice frontier over the core semantic universe. Safe subsets repair every false equivalence in the scope; unsafe subsets retain a grounded counter-witness. The all-projection row is the strongest statement: over the core field subsets, unsafe regions retain grounded counter-witnesses, every singleton is insufficient for all projections, and layer+placement is a minimum safe basis for all 498 false witnesses. Thus the repair is not a hand-picked explanation for one table row; it is the frontier of the finite projection kernel.

Table 21 shows why the repair fields are query-processing fields. Placement separates local execution, source delegation, distributed workers, and cloud-service execution. Layer and

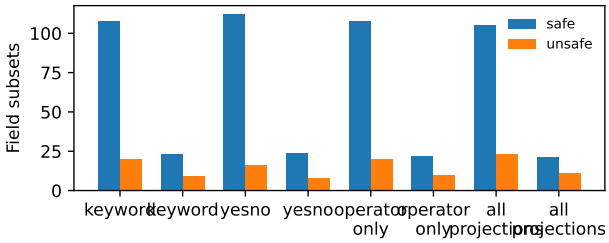

Figure 7: Semantic frontier over the core field lattice. Safe subsets repair all false equivalences in a scope; unsafe subsets retain a grounded counter-witness.

Table 25: False-portability mechanisms and representative hotspots.

Mechanism	W.	Hotspot	False
Action variant	498	Keyword BQ–CH	203
Placement	466	Keyword BQ–Trino	45
Optimizer layer	430	Keyword CH–Trino	6
Plan evidence	366	Yes/No CH–Trino	6
Decision time	362	Operator CH–Trino	123
Operator family	281	Operator Spark–Trino	115

decision time separate logical rewrite, physical planning, distributed planning, and runtime adaptation. Operator and observability separate plan operators from plan evidence, preventing logical plans, physical estimates, and runtime profiles from being treated as interchangeable. The attribution results agree with the repair certificates: the highest-attribution fields are the same fields that repair the false equivalences. A fixed layer+placement semantic basis also resolves every false equivalence within each observed engine-pair hotspot, ruling out a repair that only works in the aggregate or only through fine-grained variant identifiers.

False portability has mechanisms and hotspots. Table 25 groups false-equivalence witnesses into optimizer mechanisms and lists the largest engine-pair hotspots. The dominant mechanisms are action-variant, execution-placement, optimizer-layer, plan-evidence, and decision-time collisions. The hotspots are not random; they occur where public contracts expose different query-processing surfaces, such as source delegation versus local planning, runtime adaptivity versus compile-time planning, and stage-graph observability versus estimated local plans.

Optimizer contracts are workload-sensitive. Table 26 reports controlled feature-toggle movements. The largest movements involve CTE reuse, source boundary, statistics need, distribution triggers, and adaptivity triggers. Optimizer-contract portability is therefore not an engine-level scalar; it is a relation between engine contracts and query structure.

Table 26: Largest workload-feature-induced contract movements.

Engine	Feature	Mean dist.	Nonzero
DuckDB	reuse_structure	0.827	0.854
Trino	statistics_need	0.742	0.853
DuckDB	source_boundary	0.590	0.623
Trino	distribution_trigger	0.519	0.861
Spark SQL	adaptivity_trigger	0.442	0.858
BigQuery	distribution_trigger	0.373	0.891
Spark SQL	join_type	0.369	0.810
ClickHouse	order_limit	0.351	0.511

The evaluation closes the loop between theory and evidence. The projection-kernel theorem says that false portability is the nontrivial kernel induced by a lossy comparison vocabulary, and the grounded evaluation shows that this kernel is populated by public optimizer evidence rather than synthetic examples. The information profile quantifies distinguishability loss, the frontier analysis identifies the minimum fields that make the kernel trivial, and SQL validation checks the generated denominator on deterministic catalogs plus DuckDB and PostgreSQL engine runs. OptSem-C is therefore not a richer checklist; it is a semantics that makes optimizer-portability claims testable, diagnosable, and repairable.

6 GROUNDED CASE STUDIES

Table 27 summarizes four grounded case families. Each isolates the optimizer-semantics field that repairs a recurring false-portability mechanism.

Table 27: Grounded case studies used to interpret the false-equivalence results.

Case	Rules	Engines	Main lesson
Pushdown family	16	2	operator class matters
Plan evidence	16	4	observability is non-isomorphic
Decision time	16	3	runtime is not compile-time
Materialization	16	2	guards control equivalence

Pushdown is an operator family. Predicate movement, aggregation delegation, join delegation, limit delegation, and Top-N delegation are separate contracts; placement separates local rewrite from source delegation. **Plan evidence is not isomorphic.** Logical plans, estimated physical plans, runtime profiles, and distributed stage graphs are different public evidence states, represented by observability and decision time. **Decision time changes semantics.** Compile-time join selection, stage-boundary repartitioning, and runtime adaptive execution are not variants of the same behavior [21, 24, 35, 39, 60, 74]. **Materialization and reuse are guarded.** CTE reuse, volatility, grouped aggregation, and explicit materialization controls decide whether an intermediate is folded, materialized, or blocks pushdown. Thus operator and placement repair pushdown collisions; observability and decision time repair plan-evidence collisions; guard and variant repair reuse and materialization collisions.

7 IMPLICATIONS AND RELATED WORK

Optimizer interfaces as semantic interfaces. A public optimizer claim should name the compared action, guard, optimizer layer, execution placement, decision time, and observable evidence. These fields do not expose proprietary cost formulas; they state whether two engines are being compared on the same query-processing phenomenon. In heterogeneous analytics, where optimizers span local engines, adapters, remote sources, and service-side stages [28, 31, 40, 41, 44], the boundary is decisive: a speedup or plan difference may reflect local rewrite, source delegation, exchange placement, runtime adaptation, or a different evidence surface.

Relation to prior work. Classical optimizers separate logical equivalence, physical alternatives, access paths, search, costing, and runtime adaptation [9, 47–49, 57, 69]. OptSem-C asks which of those distinctions are public enough to support cross-engine comparison. SQL-equivalence systems decide whether expressions preserve results [1, 8, 13, 14, 72]; OptSem-C studies a relation in which result-equivalent queries can expose non-equivalent optimizer contracts. Provenance work shows why support should attach to the object being compared [7, 11, 50, 51]. Exact-cardinality testing, JOB, adaptive processing, learned optimizers, and workload benchmarks similarly sharpen denominators when older metrics hide the behavior of interest [2, 3, 5, 10, 12, 18–20, 33, 34, 37, 54, 59, 61–64, 67, 71, 76]. OptSemBench-C follows that discipline but evaluates optimizer-decision feature interactions rather than a single latency score.

Protocol and scope. A comparative study can use OptSem-C as a calibration step: state the public contract, guard, and semantic fields before interpreting speed or plan quality. If exact signatures agree, measurements have a clear denominator. If a projection collapses unequal signatures, the portability claim must name the missing field or carry a repair certificate. The scope is deliberately public. OptSem-C does not infer private cost models, prove engine-internal equivalence, or replace latency evaluation; it makes optimizer-portability claims stateable, falsifiable, and repairable at the semantic resolution exposed to users.

8 CONCLUSION

A coarse optimizer comparison can fabricate portability even when every individual system statement is public and accurate. OptSem-C formalizes this failure as a projection kernel over guarded public optimizer contracts, then grounds the kernel in public evidence and executable probes.

Keyword projection declares 2,284 equivalences, 254 of which exact contracts refute; operator-only projection declares 2,268 equivalences, 238 of which are false. Placement repairs keyword collisions, layer repairs operator-only collisions, and layer plus placement repairs all 498 headline witnesses when fine-grained action-variant labels are excluded. Speed, cost, and plan-quality comparisons are meaningful only after that public contract surface is preserved.

REFERENCES

[1] A. V. Aho, Y. Sagiv, and J. D. Ullman. 1979. Equivalences among Relational Expressions. *SIAM J. Comput.* (1979). https://doi.org/10.1137/0208032

[2] T. G. Armstrong, V. Ponnekanti, D. Borthakur, and M. Callaghan. 2013. LinkBench: A Database Benchmark Based on the Facebook Social Graph. *SIGMOD* (2013). https://doi.org/10.1145/2463676.2465296

[3] R. Avnur and J. M. Hellerstein. 2000. Eddies: Continuously Adaptive Query Processing. *SIGMOD* (2000). https://doi.org/10.1145/342009.335420

[4] E. Begoli, J. Camacho-Rodriguez, J. Hyde, M. J. Mior, and D. Lemire. 2018. Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources. *SIGMOD* (2018). https://doi.org/10.1145/3183713.3190662

[5] D. Bitton, D. J. DeWitt, and C. Turbyfill. 1983. Benchmarking Database Systems: A Systematic Approach. *VLDB* (1983). https://www.vldb.org/conf/1983/P008.PDF

[6] P. A. Boncz, M. Zukowski, and N. Nes. 2005. MonetDB/X100: Hyper-Pipelining Query Execution. *CIDR* (2005). https://www.cidrdb.org/cidr2005/papers/P19.pdf

[7] P. Buneman, S. Khanna, and W.-C. Tan. 2001. Why and Where: A Characterization of Data Provenance. *ICDT* (2001). https://doi.org/10.1007/3-540-44503-X_20

[8] A. K. Chandra and P. M. Merlin. 1977. Optimal Implementation of Conjunctive Queries in Relational Data Bases. *STOC* (1977). https://doi.org/10.1145/800105.803397

[9] S. Chaudhuri. 1998. An Overview of Query Optimization in Relational Systems. *PODS* (1998). https://doi.org/10.1145/275487.275492

[10] S. Chaudhuri, V. R. Narasayya, and R. Ramamurthy. 2009. Exact Cardinality Query Optimization for Optimizer Testing. *PVLDB* (2009). https://doi.org/10.14778/1687627.1687739

[11] J. Cheney, L. Chiticariu, and W.-C. Tan. 2009. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases* (2009). https://doi.org/10.1561/1900000006

[12] Y. Chronis, Y. Wang, Y. Gan, S. Abu-El-Haija, C. Lin, C. Binnig, and F. Ozcan. 2024. CardBench: A Benchmark for Learned Cardinality Estimation in Relational Databases. arXiv. https://arxiv.org/abs/2408.16170

[13] S. Chu, B. Murphy, J. Roesch, A. Cheung, and D. Suciu. 2018. Axiomatic Foundations and Algorithms for Deciding Semantic Equivalences of SQL Queries. *PVLDB* (2018). https://doi.org/10.14778/3236187.3236200

[14] S. Chu, C. Wang, K. Weitz, and A. Cheung. 2017. Cosette: An Automated Prover for SQL. *CIDR* (2017). https://www.cidrdb.org/cidr2017/papers/p51-chu-cidr17.pdf

[15] Google Cloud. 2026. BigQuery Documentation: Query Plan Explanation. Documentation. https://cloud.google.com/bigquery/docs/query-plan-explanation

[16] E. F. Codd. 1970. A Relational Model of Data for Large Shared Data Banks. *Commun. ACM* (1970). https://doi.org/10.1145/362384.362685

[17] R. L. Cole and G. Graefe. 1994. Optimization of Dynamic Query Evaluation Plans. *SIGMOD* (1994). https://doi.org/10.1145/191839.191899

[18] Transaction Processing Performance Council. 2024. TPC Benchmark DS Standard Specification. TPC. https://www.tpc.org/tpcds/

[19] Transaction Processing Performance Council. 2024. TPC Benchmark H Standard Specification. TPC. https://www.tpc.org/tpch/

[20] A. Deshpande, Z. Ives, and V. Raman. 2007. Adaptive Query Processing. *Foundations and Trends in Databases* (2007). https://doi.org/10.1561/1900000001

[21] A. Dutt, C. Wang, A. Nazi, S. Kandula, V. Narasayya, and S. Chaudhuri. 2019. Selectivity Estimation for Range Predicates Using Lightweight Models. *PVLDB* (2019). https://doi.org/10.14778/3342263.3342635

[22] A. Gupta et al. 2015. Amazon Redshift and the Case for Simpler Data Warehouses. *SIGMOD* (2015). https://doi.org/10.1145/2723372.2742795

[23] A. Kipf et al. 2019. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. *CIDR* (2019). https://www.cidrdb.org/cidr2019/papers/p101-kipf-cidr19.pdf

[24] A. Pavlo et al. 2017. Self-Driving Database Management Systems. *CIDR* (2017). http://cidrdb.org/cidr2017/papers/p42-pavlo-cidr17.pdf

[25] B. Dageville et al. 2016. The Snowflake Elastic Data Warehouse. *SIGMOD* (2016). https://doi.org/10.1145/2882903.2903741

[26] D. Justen et al. 2024. POLAR: Adaptive and Non-invasive Join Order Selection via Plans of Least Resistance. *PVLDB* (2024). https://doi.org/10.14778/3648160.3648175

[27] J. Shute et al. 2013. F1: A Distributed SQL Database That Scales. *PVLDB* (2013). https://doi.org/10.14778/2536222.2536232

[28] L. M. Haas et al. 1995. Garlic: A Heterogeneous Multimedia Information System. *VLDB* (1995). https://www.vldb.org/conf/1995/P491.PDF

[29] M. Armbrust et al. 2015. Spark SQL: Relational Data Processing in Spark. *SIGMOD* (2015). https://doi.org/10.1145/2723372.2742797

[30] M. A. Soliman et al. 2014. Orca: A Modular Query Optimizer Architecture for Big Data. *SIGMOD* (2014). https://doi.org/10.1145/2588555.2595637

[31] M. Stonebraker et al. 2015. The BigDAWG Polystore System. *SIGMOD Record* (2015). https://doi.org/10.1145/2814710.2814713

[32] M. Zaharia et al. 2012. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. *NSDI* (2012). https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia

[33] O. Erling et al. 2015. The LDBC Social Network Benchmark: Interactive Workload. *SIGMOD* (2015). https://doi.org/10.1145/2723372.2742786

[34] R. Marcus et al. 2021. Bao: Making Learned Query Optimization Practical. *SIGMOD* (2021). https://doi.org/10.1145/3448016.3452838

[35] S. Krishnan et al. 2018. Learning to Optimize Join Queries With Deep Reinforcement Learning. arXiv. https://arxiv.org/abs/1808.03196

[36] S. Melnik et al. 2010. Dremel: Interactive Analysis of Web-Scale Datasets. *PVLDB* (2010). https://doi.org/10.14778/1920841.1920886

[37] Y. Deng et al. 2024. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes. *PVLDB* (2024). https://doi.org/10.14778/3659437.3659448

[38] Y. Han et al. 2021. Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation. *PVLDB* (2021). https://doi.org/10.14778/3503585.3503586

[39] Z. Yang et al. 2022. Balsa: Learning a Query Optimizer Without Expert Demonstrations. *SIGMOD* (2022). https://doi.org/10.1145/3514221.3526125

[40] Apache Software Foundation. 2026. Apache Calcite Documentation: Adapter and Optimizer Framework. Documentation. https://calcite.apache.org/docs/

[41] Apache Software Foundation. 2026. Apache DataFusion Documentation: Query Optimizer. Documentation. https://datafusion.apache.org/user-guide/explain-usage.html

[42] Apache Software Foundation. 2026. Apache Spark SQL Performance Tuning. Documentation. https://spark.apache.org/docs/latest/sql-performance-tuning.html

[43] DuckDB Foundation. 2026. DuckDB Documentation: WITH Clause and CTE Materialization. Documentation. https://duckdb.org/docs/stable/sql/query_syntax/with.html

[44] Presto Foundation. 2026. Presto Documentation: Cost-Based Optimizer. Documentation. https://prestodb.io/docs/current/optimizer/cost-based-optimizations.html

[45] Trino Software Foundation. 2026. Trino Documentation: Cost-Based Optimizations and Pushdown. Documentation. https://trino.io/docs/current/optimizer/cost-based-optimizations.html

[46] C. A. Galindo-Legaria and A. Rosenthal. 1992. Outerjoin Simplification and Reordering for Query Optimization. *SIGMOD* (1992). https://doi.org/10.1145/130283.130325

[47] G. Graefe. 1990. Encapsulation of Parallelism in the Volcano Query Processing System. *SIGMOD* (1990). https://doi.org/10.1145/93597.98720

[48] G. Graefe. 1995. The Cascades Framework for Query Optimization. *IEEE Data Engineering Bulletin* (1995). https://www.sigmod.org/publications/dblp/db/journals/debu/Graefe95a.html

[49] G. Graefe and W. J. McKenna. 1993. The Volcano Optimizer Generator: Extensibility and Efficient Search. *ICDE* (1993). https://doi.org/10.1109/ICDE.1993.344061

[50] T. J. Green. 2009. Containment of Conjunctive Queries on Annotated Relations. *ICDT* (2009). https://doi.org/10.1145/1514894.

1514915
[51] T. J. Green, G. Karvounarakis, and V. Tannen. 2007. Provenance Semirings. *PODS* (2007). https://doi.org/10.1145/1265530.1265535
[52] PostgreSQL Global Development Group. 2026. PostgreSQL Documentation: Using EXPLAIN and Query Planning. Documentation. https://www.postgresql.org/docs/current/using-explain.html
[53] B. Hilprecht, A. Schmidt, M. Kulessa, A. Molina, K. Kersting, and C. Binnig. 2020. DeepDB: Learn from Data, Not from Queries! *PVLDB* (2020). https://doi.org/10.14778/3407790.3407793
[54] ClickHouse Inc. 2024. ClickBench: A Benchmark for Analytical DBMS. Online benchmark. https://benchmark.clickhouse.com/
[55] ClickHouse Inc. 2026. ClickHouse Documentation: JOIN Optimization and Algorithms. Documentation. https://clickhouse.com/docs/guides/joining-tables
[56] Snowflake Inc. 2026. Snowflake Documentation: EXPLAIN. Documentation. https://docs.snowflake.com/en/sql-reference/sql/explain
[57] Y. E. Ioannidis. 1996. Query Optimization. *Comput. Surveys* (1996). https://doi.org/10.1145/235968.235970
[58] Z. G. Ives, D. Florescu, M. Friedman, A. Levy, and D. S. Weld. 1999. An Adaptive Query Execution System for Data Integration. *SIGMOD* (1999). https://doi.org/10.1145/304182.304205
[59] N. Kabra and D. J. DeWitt. 1998. Efficient Mid-Query Re-Optimization of Sub-Optimal Query Execution Plans. *SIGMOD* (1998). https://doi.org/10.1145/276304.276337
[60] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis. 2018. The Case for Learned Index Structures. *SIGMOD* (2018). https://doi.org/10.1145/3183713.3196909
[61] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. 2015. How Good Are Query Optimizers, Really? *PVLDB* (2015). https://doi.org/10.14778/2850583.2850594
[62] V. Leis, B. Radke, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. 2018. Query Optimization Through the Looking Glass, and What We Found Running the Join Order Benchmark. *VLDB Journal* (2018). https://doi.org/10.1007/s00778-017-0480-7
[63] R. Marcus and O. Papaemmanouil. 2019. Neo: A Learned Query Optimizer. *PVLDB* (2019). https://doi.org/10.14778/3342263.3342644
[64] V. Markl, V. Raman, D. Simmen, G. Lohman, H. Pirahesh, and M. Cilimdzic. 2004. Robust Query Processing Through Progressive Optimization. *SIGMOD* (2004). https://doi.org/10.1145/1007568.1007642
[65] G. Moerkotte and T. Neumann. 2006. Analysis of Two Existing and One New Dynamic Programming Algorithm for the Generation of Optimal Bushy Join Trees without Cross Products. *VLDB* (2006). https://dblp.org/rec/conf/vldb/MoerkotteN06.html
[66] T. Neumann. 2011. Efficiently Compiling Efficient Query Plans for Modern Hardware. *PVLDB* (2011). https://doi.org/10.14778/2002938.2002940
[67] P. ONeil, E. ONeil, X. Chen, and S. Revilak. 2009. The Star Schema Benchmark and Augmented Fact Table Indexing. *TPCTC* (2009). https://www.cs.umb.edu/~poneil/StarSchemaB.PDF
[68] M. Raasveldt and H. Muehleisen. 2019. DuckDB: An Embeddable Analytical Database. *SIGMOD* (2019). https://doi.org/10.1145/3299869.3320212
[69] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access Path Selection in a Relational Database Management System. *SIGMOD* (1979). https://doi.org/10.1145/582095.582099
[70] M. Steinbrunn, G. Moerkotte, and A. Kemper. 1997. Heuristic and Randomized Optimization for the Join Ordering Problem. *VLDB Journal* (1997). https://doi.org/10.1007/s007780050048
[71] C. Turbyfill, C. Orji, and D. Bitton. 1989. AS3AP: An ANSI SQL Standard Scaleable and Portable Benchmark for Relational Database Systems. *VLDB* (1989). https://www.vldb.org/conf/1989/P159.PDF
[72] S. Wang, S. Pan, and A. Cheung. 2024. QED: A Powerful Query Equivalence Decider for SQL. *PVLDB* (2024). https://doi.org/10.14778/3681954.3682024
[73] Z. Wu, P. Negi, M. Alizadeh, T. Kraska, and S. Madden. 2023. FactorJoin: A New Cardinality Estimation Framework for Join Queries. *SIGMOD* (2023). https://arxiv.org/abs/2212.05526
[74] J. Yang, N. Bruno, P. Sen, Y. Chen, and S. Chaudhuri. 2020. Selectivity Estimation with Deep Likelihood Models. *SIGMOD* (2020). https://doi.org/10.1145/3318464.3389721
[75] Z. Yang, E. Liang, A. Kamsetty, C. Wu, Y. Duan, X. Chen, P. Abbeel, J. M. Hellerstein, S. Krishnan, and I. Stoica. 2020. Deep Unsupervised Cardinality Estimation. *PVLDB* (2020). https://doi.org/10.14778/3407790.3407794
[76] R. Zhu, W. Chen, B. Ding, X. Chen, A. Pfadler, Z. Wu, and J. Zhou. 2023. Lero: A Learning-to-Rank Query Optimizer. *PVLDB* (2023). https://doi.org/10.14778/3583140.3583164